2.5 Hands-on Lab: Pull, modify, and run a Deep Learning container from NGC

What you'll do

1. Log into NGC from the CLI
2. Pull a GPU-optimized deep learning container (PyTorch example)
3. Run it with GPU access and a mounted workspace
4. Modify it (install extra deps)
5. (Optional) Bake your changes into a new image

Prereqs (5–10 min)

- A GPU machine with **NVIDIA driver + CUDA** working (`nvidia-smi` shows your GPU).
- **Docker** + **NVIDIA Container Toolkit** installed (so `docker run --gpus all …` works).
- An **NGC account** and **API key** (free): create one at ngc.nvidia.com → *Setup* → *API Key*.

> Tip: If you did Module 1.5, you're already set.

Part A — Install & configure NGC CLI (2–3 min)

1. **Download NGC CLI** (Linux/macOS):

```
curl -O https://ngc.nvidia.com/downloads/ngccli_linux.zip
unzip ngccli_linux.zip
sudo mv ngc-cli/ngc /usr/local/bin/
ngc --version
```

2. **Configure with your API key**:

```
ngc config set
# Paste your API key when prompted. Accept defaults for org/team unless you use them.
```

3. **Sanity check**:

```
ngc registry list
```

Part B — Find and pull the container you want (2–4 min)

1. **List PyTorch images/tags**:

```
ngc registry image list nvidia/pytorch
```

2. **Pick a specific tag** (avoid `latest`). Example:

```
export PT_TAG=24.06-py3     # replace with a tag you saw in the list
```

3. **Pull via Docker (from NGC's nvcr.io)**:

```
docker pull nvcr.io/nvidia/pytorch:${PT_TAG}
```

(You can do the same for TensorFlow, RAPIDS, Triton, etc. Swap `nvidia/pytorch` for the repo you need.)

Part C — Run the container with GPU access + a workspace (3–6 min)

1. **Create a workspace directory** on the host:

```
mkdir -p ~/ngc-work
```

2. **Run interactively with best-practice flags**:

```
docker run --rm -it --gpus all \
  --ipc=host --shm-size=16g \
  -v ~/ngc-work:/workspace \
  -p 8888:8888 \
  nvcr.io/nvidia/pytorch:${PT_TAG} /bin/bash
```

3. **Inside the container, verify GPU**:

```
nvidia-smi
python - << 'PY'
import torch
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
PY
```

4. **(Optional) Launch Jupyter Lab inside the container**:

```
jupyter lab --ip=0.0.0.0 --no-browser --allow-root --NotebookApp.token=''
```

Open `http://<YOUR_HOST_IP>:8888` in a browser. Anything you create under `/workspace` is persisted to `~/ngc-work` on the host.

Part D — Modify the running container (1–5 min)

Quick changes (ad-hoc) — install extra libs:

```
pip install transformers accelerate lightning==2.* opencv-python
```

Test they work, save code under `/workspace`, commit results/models to your repo or volume.

Part E — Bake changes into your own image (recommended) (3–8 min)

1. **Create a Dockerfile** on the host (in `~/ngc-work`):

```
# Dockerfile.ngc-pytorch
FROM nvcr.io/nvidia/pytorch:24.06-py3  # or your chosen tag
WORKDIR /workspace

# System deps (example)
RUN apt-get update && apt-get install -y git && rm -rf /var/lib/apt/lists/*

# Python deps
RUN pip install --no-cache-dir transformers accelerate lightning==2.* opencv-python

# Optional: create a non-root user
# RUN useradd -m app && chown -R app:app /workspace
# USER app

CMD ["/bin/bash"]
```

2. **Build the custom image**:

```
cd ~/ngc-work
docker build -f Dockerfile.ngc-pytorch -t my-ngc/pytorch:custom .
```

3. **Run your custom image**:

```
docker run --rm -it --gpus all \
  --ipc=host --shm-size=16g \
  -v ~/ngc-work:/workspace \
  -p 8888:8888 \
  my-ngc/pytorch:custom
```

> Now your tools are baked in; anyone on your team can reuse this exact environment.

Part F — (Optional) Export for fast inference

If you fine-tuned a model and want speed:

- **Export to ONNX/TensorRT** (varies by model), then
- Serve with **Triton Inference Server** (also on NGC).

Example (conceptual):

```
# Inside container, export to ONNX (model-specific code)
python export_to_onnx.py --checkpoint /workspace/ckpt.pt --out /workspace/model.onnx
```

```
# Convert to TensorRT (trtexec or Python API)
trtexec --onnx=/workspace/model.onnx --saveEngine=/workspace/model.plan
```

Part G — (Optional) Push to a registry

If you use a private registry (e.g., ECR/GCR/ACR or self-hosted):

```
# Tag for your registry
docker tag my-ngc/pytorch:custom <YOUR_REGISTRY>/my-ngc/pytorch:custom
docker login <YOUR_REGISTRY>
docker push <YOUR_REGISTRY>/my-ngc/pytorch:custom
```

Clean up

- Stop any running containers (`Ctrl+C` if interactive).
- Remove images you don't need:

```
docker images
docker rmi <IMAGE_ID>
```

Troubleshooting quick hits

- **`docker: Error response from daemon: could not select device driver "" with capabilities: [[gpu]]`**
  → NVIDIA Container Toolkit not installed or misconfigured. Reinstall and restart Docker.
- **`nvidia-smi` works on host but not in container**
  → Missing `--gpus all` flag or runtime not configured. Re-run with `--gpus all` .
- **Pip installs fail**
  → Add `--no-cache-dir` , ensure network egress, or update `pip` (`python -m pip install -U pip`).
- **Jupyter not reachable**
  → Ensure you mapped `-p 8888:8888` and your security group/firewall allows that port (or use SSH tunneling).
